

Lab 1. Python Fundamentals

Tomasz Martyński

Task 1. (sum of all numbers that...)

Write a function `sum_multiples(n)` that calculates the sum of all numbers less than n that are divisible by 3 or 5.

Example: `sum_multiples(10)` # $3 + 5 + 6 + 9 = 23$

```
In [1]: def sum_multiples(n):  
        all_sum=0  
        for i in range(n):  
            if i%3==0 or i%5==0:  
                all_sum+=i  
        return(all_sum)  
  
print(sum_multiples(10))
```

23

Task 2. (Collatz rules)

For a given number n , apply the Collatz rules:

- If the number is even, divide it by 2
- If the number is odd, apply $3n + 1$

Repeat until the number reaches 1. Return the number of steps.

Example: `collatz_length(6)` returns 8 because $6 \rightarrow 3 \rightarrow 10 \rightarrow 5 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$

```
In [2]: def collatz_length(n):
        num_steps=0
        while n!=1:
            if n%2==0:
                n=int(n/2)
            else:
                n=3*n+1
            num_steps+=1
        return(num_steps)

print(collatz_length(6))
```

8

Task 3. (fruits array)

Create a list with the values

```
fruits = ['apple', 'banana', 'cherry', 'apple', 'banana', 'apple', 'orange', 'cherry']
```

1. Print the first and last item.
2. Add 'pear' to the end.
3. Count how many times 'apple' appears in the list.
4. Change 'apple' to 'pear'.
5. Find the index of the second occurrence of 'banana'.

```
In [6]: fruits = ['apple', 'banana', 'cherry', 'apple', 'banana', 'apple', 'orange', 'cherry']
#1
print(f"First item: {fruits[0]}, last item: {fruits[-1]}.")

#2
fruits.append("pear")
print(fruits)
```

```

#3
print(fruits.count("apple"))

#4
fruits=["pear" if fruit=="apple" else fruit for fruit in fruits]
print(fruits)

#5
first_occurrence=fruits.index("banana")
fruits.index("banana", first_occurrence+1)

```

First item: apple, last item: cherry.

```
['apple', 'banana', 'cherry', 'apple', 'banana', 'apple', 'orange', 'cherry', 'pear']
```

```
3
```

```
['pear', 'banana', 'cherry', 'pear', 'banana', 'pear', 'orange', 'cherry', 'pear']
```

Out[6]: 4

Task 4. (dictionary by combining lists)

You are given two lists:

```

keys = ["name", "age", "country"]
values = ["Tom", 30, "Canada"]

```

Create a dictionary by combining them

Expected result:

```
{"name": "Tom", "age": 30, "country": "Canada"}
```

```

In [4]: keys = ["name", "age", "country"]
values = ["Tom", 30, "Canada"]
combined_dict=dict(zip(keys, values))
print(combined_dict)

```

```
{'name': 'Tom', 'age': 30, 'country': 'Canada'}
```

Task 5. (fruits array again)

Create a list with the values

```
fruits = ['apple', 'banana', 'cherry', 'apple', 'banana', 'apple', 'orange', 'cherry']
```

1. Build a frequency dictionary showing how many times each fruit appears. Expected output: `{'apple': 3, 'banana': 2, 'cherry': 2, 'orange': 1}`.
2. Make a list of fruits that occur more than once.

```
In [7]: fruits=['apple', 'banana', 'cherry', 'apple', 'banana', 'apple', 'orange', 'cherry']
#1
freq_dict={fruit : fruits.count(fruit) for fruit in fruits}
print(freq_dict)
#2
popular_fruits=[fruit for fruit in freq_dict if freq_dict[fruit]>1]
print(popular_fruits)
```

```
{'apple': 3, 'banana': 2, 'cherry': 2, 'orange': 1}
['apple', 'banana', 'cherry']
```

Task 6. (classify score)

Write a function `classify_score(score)` that:

1. Accepts an integer between 0 and 100
2. Returns:
 - "Excellent" if score ≥ 90
 - "Good" if score ≥ 75
 - "Pass" if score ≥ 60

- "Fail" otherwise

Then, use this function to classify a list of scores

```
scores = [95, 82, 67, 58, 74, 99, 45]
```

Return the result as a list of tuples in the format: (score, category)

Expected output:

```
[(95, 'Excellent'), (82, 'Good'), (67, 'Pass'), (58, 'Fail'), (74, 'Pass'), (99, 'Excellent'), (45, 'Fail')]
```

```
In [8]: def classify_score(score):
        return (score>=90)*"Excellent"+(90>score>=75)*"Good"+(75>score>=60)*"Pass"+(60>score)*"Fail"    #could be cases here, but

scores=[95, 82, 67, 58, 74, 99, 45]
results=[(mark, classify_score(mark)) for mark in scores]
print(results)
```

```
[(95, 'Excellent'), (82, 'Good'), (67, 'Pass'), (58, 'Fail'), (74, 'Pass'), (99, 'Excellent'), (45, 'Fail')]
```

Task 7. (average of the grades)

For a given list of dictionaries representing students with grades

```
students = [
    {"name": "Alice", "grades": [80, 90, 85]},
    {"name": "Bob", "grades": [60, 70]},
    {"name": "Eva", "grades": []}
]
```

write a function that adds a new key "avg" to each dictionary with the average of the grades. If the list of grades is empty, set "avg" to None .

Expected Output:

```
[{'name': 'Alice', 'grades': [80, 90, 85], 'avg': 85.0}, {'name': 'Bob', 'grades': [60, 70], 'avg': 65.0},  
{ 'name': 'Eva', 'grades': [], 'avg': None}]
```

```
In [7]: students = [  
    {"name": "Alice", "grades": [80, 90, 85]},  
    {"name": "Bob", "grades": [60, 70]},  
    {"name": "Eva", "grades": []}  
]  
  
def average_calculator(graded_people):  
    for person in graded_people:  
        average=None  
        if person["grades"]!=[]:  
            average=sum(person["grades"])/len(person["grades"])  
            person["avg"]=average  
  
average_calculator(students)  
print(students)
```

```
[{'name': 'Alice', 'grades': [80, 90, 85], 'avg': 85.0}, {'name': 'Bob', 'grades': [60, 70], 'avg': 65.0}, {'name': 'Eva', 'grades': [], 'avg': None}]
```

Challenge (smart grocery store inventory system)

This task is not mandatory!

Your task is to build a mini inventory management program for a grocery store using core Python only (no external libraries). The store sells items in different categories: fruits, vegetables, bakery, and beverages.

Scenario:

- Each product has: name (`str`), price (`float`), stock quantity (`int`), category (str).
- Customers buy multiple items at once, and you need to process their orders, update stock, and compute totals.
- If a product is missing or the quantity is too high, the program should handle it gracefully (with exceptions).
- The store also wants daily statistics: which category sold most, total revenue, and any low-stock alerts.

Do the following steps:

1. Define the inventory with data:

```
inventory = {
    "apple": {"price": 1.20, "stock": 50, "category": "fruit"},
    "banana": {"price": 0.80, "stock": 60, "category": "fruit"},
    "orange": {"price": 1.00, "stock": 40, "category": "fruit"},
    "bread": {"price": 2.50, "stock": 20, "category": "bakery"},
    "croissant": {"price": 1.80, "stock": 15, "category": "bakery"},
    "milk": {"price": 1.50, "stock": 30, "category": "beverage"},
    "juice": {"price": 2.20, "stock": 25, "category": "beverage"},
    "carrot": {"price": 0.60, "stock": 70, "category": "vegetable"},
    "tomato": {"price": 0.90, "stock": 55, "category": "vegetable"},
    "potato": {"price": 0.50, "stock": 100, "category": "vegetable"}
}
```

2. Implement a function `customer_card(shopping_cart, inventory)` that processes a customer's shopping cart which is a list of tuples, having a form `(product_name, quantity)`. The function should:

- Verify product availability:
 - Check if each product exists in the store's inventory. If a product is missing, raise an appropriate error (using function `raise()`).
- Check stock levels:
 - Ensure there is enough stock to fulfill the requested quantity.
 - If stock is insufficient, raise an error.
- Update the inventory:
 - Deduct the purchased quantities from the stock.
- Calculate total price:
 - Multiply quantities by prices and sum to compute the final cost of the cart.
- Return the total price:

- The function should return the total amount the customer must pay.

Finally, test the function with different shopping carts. For example you should obtain:

1. `cart1 = [("apple", 5), ("banana", 10), ("milk", 2)]`

As a result: `17.0`

2. `cart2 = [("apple", 5), ("grapes", 3)]`

As a result: `ValueError: Product 'grapes' not found in inventory.`

3. `cart3 = [("orange", 50), ("bread", 5)]`

As a result: `ValueError: Insufficient stock for 'orange'. Requested: 50, Available: 40`

```
In [29]: inventory = {
    "apple": {"price": 1.20, "stock": 50, "category": "fruit"},
    "banana": {"price": 0.80, "stock": 60, "category": "fruit"},
    "orange": {"price": 1.00, "stock": 40, "category": "fruit"},
    "bread": {"price": 2.50, "stock": 20, "category": "bakery"},
    "croissant": {"price": 1.80, "stock": 15, "category": "bakery"},
    "milk": {"price": 1.50, "stock": 30, "category": "beverage"},
    "juice": {"price": 2.20, "stock": 25, "category": "beverage"},
    "carrot": {"price": 0.60, "stock": 70, "category": "vegetable"},
    "tomato": {"price": 0.90, "stock": 55, "category": "vegetable"},
    "potato": {"price": 0.50, "stock": 100, "category": "vegetable"}
}

def customer_card(shopping_cart, inventory):
    total_cost=0
    for item in shopping_cart:
        if item[0] not in inventory.keys():
            raise ValueError(f"Product '{item[0]}' not found in inventory.")
        avail_product=inventory[item[0]]
        if avail_product["stock"]<item[1]:
            raise ValueError(f"Insufficient stock for '{item[0]}'. Requested: {item[1]}, Available: {avail_product['stock']}")
```



```

    #everything is okay and we can start shopping
    avail_product["stock"]-=item[1]
    total_cost+=item[1]*avail_product["price"]
    return total_cost

```

Check your solution below:

```

In [30]: cart1 = [("apple", 5), ("banana", 10), ("milk", 2)]
print(customer_card(cart1, inventory))

print(f"Inventory has indeed changed - current state:")
for item in cart1:
    print(f"> {item[0]}: {inventory[item[0]]['stock']}")

```

17.0

Inventory has indeed changed - current state:

```

> apple: 45
> banana: 50
> milk: 28

```

```

In [31]: cart2 = [("apple", 5), ("grapes", 3)]
customer_card(cart2, inventory)

```

```

-----
ValueError                                Traceback (most recent call last)
Cell In[31], line 2
      1 cart2 = [("apple", 5), ("grapes", 3)]
----> 2 customer_card(cart2, inventory)

Cell In[29], line 19, in customer_card(shopping_cart, inventory)
     17 for item in shopping_cart:
     18     if item[0] not in inventory.keys():
--> 19         raise ValueError(f"Product '{item[0]}' not found in inventory.")
     20     avail_product=inventory[item[0]]
     21     if avail_product["stock"]<item[1]:

ValueError: Product 'grapes' not found in inventory.

```

```

In [32]: cart3 = [("orange", 50), ("bread", 5)]
customer_card(cart3, inventory)

```

```
-----  
ValueError                                Traceback (most recent call last)  
Cell In[32], line 2  
      1 cart3 = [("orange", 50), ("bread", 5)]  
----> 2 customer_card(cart3, inventory)  
  
Cell In[29], line 22, in customer_card(shopping_cart, inventory)  
     20 avail_product=inventory[item[0]]  
     21 if avail_product["stock"]<item[1]:  
--> 22     raise ValueError(f"Insufficient stock for '{item[0]}'. Requested: {item[1]}, Available: {avail_product['stock']}")  
     23 #everything is okay and we can start shopping  
     24 avail_product["stock"]-=item[1]  
  
ValueError: Insufficient stock for 'orange'. Requested: 50, Available: 40
```